Why can cloud operations be tricky? Well, in the context of cloud-native solutions, be that a workload running on Kubernetes or a bunch of lambda functions, what all of them have in common is many moving parts with potentially many different owners. So when end users of your cloud-native app are impacted—for example, by slower-than-usual response times or (partial) failures—the murder mystery starts: Which of the services along the request path contributed to the problem? Who owns it and, hence, should be notified to fix it?

Remember that for the end user, the fact that you're running, say, hundreds of microservices is unknown and, frankly, shouldn't matter. What they perceive is that something is not working, and it's your organization's responsibility to make sure you detect such issues—ideally, before you learn of them from your users—and have strategies in place not only to fix the issues as fast as you can but also learn from an issue to make sure it doesn't happen again. In this context, we can consider tracking the customer experience (not always clear who owns this) versus infrastructure tracking (usually clearly defined ownership). Let's start with incidents and how to detect and respond to them in a structured manner.

7.1 Incident management

In the context of this book, we'll consider two incidents: unexpected behavior of services or the platform that negatively affects its function or performance, and a repeat of unexpected behavior that is yet to have the learnings applied. For the latter case, the repeated unexpected behavior, it can be helpful to have a runbook of how previous incidents were resolved for handling the repeat situation.

Generally, it is best to have an incident response process in place. This incident response process is typically a set of procedures that define how you go about resolving and learning from an incident. You can think of it in terms of three phases:

1. First, you need to be able to detect that something is going wrong (health and performance monitoring).
2. Then, you need to be able to react to and handle an incident in a structured manner.
3. Last but not least, you need to learn from an incident. You want to avoid repeating a similar occurrence in the future (what action items should be completed to not have repeat occurrence of the same problem). We'll start by covering detecting an issue.

7.1.1 Health and performance monitoring

We previously discussed that the first step in resolving an incident is to be able to understand that something is broken in the first place. This may sound simple, but in a cloud-native system with many components, it is a nontrivial issue. In other words, manual approaches, such as visiting your website in a browser and checking if everything looks good, are not something you want to consider. Instead, aim to automate as